README

Giolit SystemVerilog Assertion Verifier 1.0: installation, first steps, commands and this package

Contents

What this is 1

This package 2

1. Prerequisites 2

2. Installation 3

3. First test 3

4. Commands 4

5. Configuration 5

6. Purchasing a license 5

7. Troubleshooting 5

8. Support 5

What this is

Formal verification of SystemVerilog assertions on RTL designs: a proof that every assertion holds for all inputs, or the clock-by-clock trace that breaks one.

The **Giolit SystemVerilog Assertion Verifier** (command `sva-verify`) reads a SystemVerilog design (IEEE 1800-2017) with its assertions and decides every `assert property`, `cover property` and immediate assertion in it, for every input sequence and every start state:

Status	Meaning
PROVED	No run of the design violates the assertion.
FAILED	A run violates it; the run is printed cycle by cycle.
COVERED / **UNREACHABLE**	A cover property: a witness run exists / no run can reach it.
REACHED	A liveness goal is reached on some run (witness shown), but not proved for every run.

Status	Meaning
UNDECIDED	Not decided within the time budget; the reason is printed.
UNSUPPORTED	The property uses a construct the verifier does not model; the reason is printed.

`assume property` constrains the inputs. Liveness properties (`s_eventually`, strong sequences, `s_until`) are proved by k-liveness or refuted by a lasso.

This is the **trial edition** (see the License Terms): 1000 runs within 50 days on one computer.

This package

Folder	Contents
`code/`	the verifier: `bin/sva-verify`, its libraries (`lib/`), the public C++ API (`include/`), the tutorial examples (`share/giolit/examples/`), the SDK examples (`share/giolit/sdk-examples/`), and `install.sh` / `uninstall.sh`
`documents/`	this README, the **Tutorial** (start there), the **Product Poster**, the **Tool Comparison**, the **License Terms** and the third-party notices
`benchmarks/`	public benchmark designs with a known answer, from the EBMC regression suite and the SymbiYosys examples, with their licenses; `MANIFEST.tsv` lists every verification (expected result, options, files)
`results/`	the verifier's result for every benchmark, in the same folders: the console output, `results.json` and `reproduce.sh`; `SUMMARY.csv` lists them all
`SHA256SUMS`	checksums of every file (`sha256sum -c SHA256SUMS`)

1. Prerequisites

Requirement	Details
Operating system	Linux x86-64. Tested on **Ubuntu 24.04 LTS**.
Memory	4 GB RAM minimum, 8 GB recommended
Runtime libraries	LLVM/Clang 18 (`libllvm18`, `libclang-cpp18`, `libclang-common-18-dev`), `libc6-dev`, OpenSSL 3 (`libssl3`)
LuaLaTeX *(PDF certificates and reports only)*	`texlive-luatex`, `texlive-latex-recommended`, `texlive-latex-extra`, `texlive-pictures`, `texlive-fonts-extra`
PDF protection *(licensed certificates)*	`qpdf` or `mupdf-tools`: with a purchased license, PDFs are password-protected

```
sudo apt update
sudo apt install libllvm18 libclang-cpp18 libclang-common-18-dev libc6-dev libssl3
sudo apt install texlive-luatex texlive-latex-recommended texlive-latex-extra \
                 texlive-pictures texlive-fonts-extra qpdf          # for --report
```

Without LuaLaTeX the verifier works fully; `--report` then writes every file of the bundle except the PDF.

2. Installation

```
tar -xzf giolit-sva-verifier-1.0.0-trial.tar.gz
cd giolit-sva-verifier-1.0.0-trial/code
./install.sh                # into ~/.local/giolit-sva-verify, command in ~/.local/bin
```

`./install.sh --system` installs into `/opt/giolit-sva-verify` (run it with `sudo`), `./install.sh --prefix DIR` anywhere else. The installer checks the libraries, copies the tool and runs a self-test. Then:

```
sva-verify --version        # sva-verify 1.0.0 (trial version)
sva-verify --license        # runs left and expiry dates (uses no run)
```

`uninstall.sh` in the installation folder removes it again; your license state in `~/.giolit/sva-verify` is kept.

3. First test

```
cd ~/.local/giolit-sva-verify/share/giolit/examples
sva-verify --reset=rst 01_counter.sv
```

```
$ sva-verify --reset=rst 01_counter.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 4 properties

  PROVED      assert    counter.p_range  (01_counter.sv:16)
  PROVED      assert    counter.p_wrap  (01_counter.sv:17)
  PROVED      assert    counter.p_hold  (01_counter.sv:18)
  COVERED     cover     counter.c_wrap  (01_counter.sv:19)
      witness, 12 cycles (cycle 0 is the first clock edge):
      cycle 0  : cnt=15 en=1 rst=1
      cycle 1  : cnt=0 rst=0
      cycle 2  : cnt=1
      cycle 3  : cnt=2
      cycle 4  : cnt=3
      cycle 5  : cnt=4
      cycle 6  : cnt=5
      cycle 7  : cnt=6
      cycle 8  : cnt=7
      cycle 9  : cnt=8
      cycle 10 : cnt=9
      cycle 11 : cnt=0

4 properties: 3 proved, 0 failed, 1 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.1 s)
Result: ALL ASSERTIONS PROVED
```

`02_counter_bug.sv` wraps at 10 instead of 9:

```
$ sva-verify --reset=rst 02_counter_bug.sv
Giolit SystemVerilog Assertion Verifier 1.0.0 — top (auto), 1 property

  FAILED      assert    counter_bug.p_range  (02_counter_bug.sv:13)
      counterexample, 12 cycles (cycle 0 is the first clock edge):
      cycle 0  : cnt=15 en=1 rst=1
      cycle 1  : cnt=0 rst=0
      cycle 2  : cnt=1
      cycle 3  : cnt=2
```

```
      cycle 4  : cnt=3
      cycle 5  : cnt=4
      cycle 6  : cnt=5
      cycle 7  : cnt=6
      cycle 8  : cnt=7
      cycle 9  : cnt=8
      cycle 10 : cnt=9
      cycle 11 : cnt=10

1 properties: 0 proved, 1 failed, 0 covered, 0 unreachable covers, 0 liveness reached (not proved), 0
    undecided, 0 unsupported  (0.0 s)
Result: ASSERTION FAILURES FOUND
```

Each cycle line shows the signals that changed. The **Tutorial** walks through all nine examples.

4. Commands

```
sva-verify [options] design.sv [more.sv ...]
```

Option	Effect
`--top=MODULE`	the top-level module (default: the only uninstantiated one)
`--reset=SIGNAL`	the reset is active in the first cycle (`rst`, or `'!rst_n'` for active low); afterwards it is a free input
`--property=NAME`	verify only this property, on its cone of influence
`-p EXPR`	also assert `always EXPR` on the top module
`-G NAME=VALUE`	override a parameter of the top module
`-I DIR`, `-D NAME[=V]`	preprocessor include directories and macros; other front-end options after `--sv-args`
`--multiclock`	exact edge semantics for several or derived clocks (default: one global clock)
`--encoding=MODE`	`hybrid` (default), `word`, `bit`, or `race` (all three in parallel)
`--timeout=SECONDS`	total time budget (default 60)
`--json=FILE`	per-property results with traces as JSON
`--report=DIR`	the evidence bundle: a certificate (PDF) with the proofs when every assertion is proved, otherwise a report; every counterexample as a waveform (VCD) and a replay testbench; a signed statement (Tutorial, section 15)
`--issued-to=NAME`	with `--report`: who the certificate is for
`--margins`	the safety margin of every numeric bound of the proved assertions and the Giolit Safety Resilience Score, 1 to 5 stars (Tutorial, section 17; included in `--report`)
`--verbose`	property source text and full traces
`--activate=FILE`	apply a purchased license (with `--name=`, `--address=`, `--password=`, or asked for)
`--license`, `--version`, `--help`	

Exit code: **0** every assertion proved, **1** an assertion fails, **2** some undecided or unsupported, **3** usage or input error, **4** no valid license.

5. Configuration

Variable	Effect	Default
`V8_MEMORY_MB`	memory limit of a run, in MB	half the available memory, at most 4096

6. Purchasing a license

Write to **business.giolitlabs@gmail.com** or visit **www.giolit.com** with the output of `sva-verify --version` and `sva-verify --license`. You receive a license file and a password; apply them with

```
sva-verify --activate customer.lic      # asks for the buyer's name, address and password
```

The License Terms, section 7, explain the details.

7. Troubleshooting

Symptom	Cause and remedy
`error while loading shared libraries: libLLVM.so.18.1`	install the libraries of section 1
`SystemVerilog errors: ...`	the design does not elaborate; pass `-I`, `-D` and `--top` as for your simulator
`unsupported SystemVerilog: ...`	a construct the verifier does not model; the message names it (Tutorial, section 13)
an assertion fails in cycle 0 or 1	registers start arbitrary: add `--reset` (Tutorial, section 3)
a property is UNDECIDED	Tutorial, section 11: more time, `--property`, `-G`, assumptions
exit code 4	the trial's runs or days are used up, the build is too old, or the license is damaged; see `sva-verify --license`
the report has no PDF (`lualatex is not installed`, or `lualatex failed`)	install the LaTeX packages of section 1; after a failure, the `.log` next to the missing PDF names the missing package
`cannot password-protect the PDF`	install `qpdf` (or `mupdf-tools`); licensed certificates are only written protected

8. Support

Giolit Labs — www.giolit.com — business.giolitlabs@gmail.com

When reporting a problem, include `sva-verify --version`, `sva-verify --license`, the command line and, if possible, the design.

Giolit Labs is the sole owner of the Giolit SystemVerilog Assertion Verifier. Use is subject to the License Terms.